Deep Learning for Space Time Graph

Assignment 2

River Forecasting on a Sensor Network, and
Motion or Memory? Stress-Testing a ConvLSTM Video Forecaster

https://github.com/mubeenahmed-ai/AI651-PA2

Mubeen Ahmed

Student ID: 25280101

Fall 2026

October 26, 2026

1 Introduction

This assignment has two parts, both of which ask how far, in space and in time, information must travel before a forecaster can use it, and whether the model's structure allows it to travel that far. Question 1 designs a system that forecasts the water level one hour ahead at four river stations every ten minutes. The river is encoded as a directed graph whose nodes are the reservoir, the four main-river stations, the monitored tributary and a latent node for the unmonitored tributary. A causal temporal encoder summarises each node's history. A lag-routing attention layer then lets each station read every upstream node at a learned delay, centred on a travel-time prior computed from river distances, and a quantile head forecasts the one-hour change in level relative to the last observation. Announced reservoir releases enter through a monotone response kernel.

Question 2 trains a ConvLSTM forecaster on Moving MNIST and tests whether it learned how digits move or what the training videos look like. At 1×, model A's error is about half that of predicting blank frames and a third that of copying the last frame. It is only approximately consistent under mirror and rotation, rotation being the harder of the two, and it breaks down already at 2× speed. Symmetry augmentation (B) closes most of the rotation gap, which identifies appearance as its cause. Speed augmentation with model A's own architecture (C) cuts the error at 2× and 3× by more than a third, while a larger-reach design (D) adds only 2% at 3×, so the speed failure was mainly unfamiliar motion rather than architectural reach. Wrapping D as a symmetrised forecaster makes it exactly consistent, at about eight times the inference cost.

The main contributions of this study are:

- **A river forecasting design (Q1)** with an explicit tensor specification, a travel-time-aware routing layer whose reach covers every upstream source in one hop, and a separate treatment of missing data, delayed data and announced and deviating reservoir releases.
- **A complete, resumable experimental pipeline (Q2)** covering all transformation checks, six tables and six figures, with both seeds reported for every learned model and the Q2.5 and Q2.11 predictions committed before the corresponding experiments.
- **A three-way separation of failure causes (Q2)**: appearance (mirror/rotation), unfamiliar motion (2× and, mostly, 3×) and architectural reach (a small residual at 3×).

All code, notebooks with outputs, figures and results are available at https://github.com/mubeenahmed-ai/AI651-PA2.

2 Question 1: River Forecasting Proposal

The service issues, every ten minutes, a forecast of the water level one hour ahead at the four main-river stations S1–S4. The proposal is a small, physically structured space–time graph network in the family of graph-based sensor forecasters [6, 7]: the *Lag-Routing Graph Forecaster*. A station's level in one hour is mostly its current level plus water already on its way from upstream, so the model must (i) know the river's topology, (ii) look back upstream by a delay close to the travel time, and (iii) let that delay change with flow conditions.

2.1 Architecture diagram

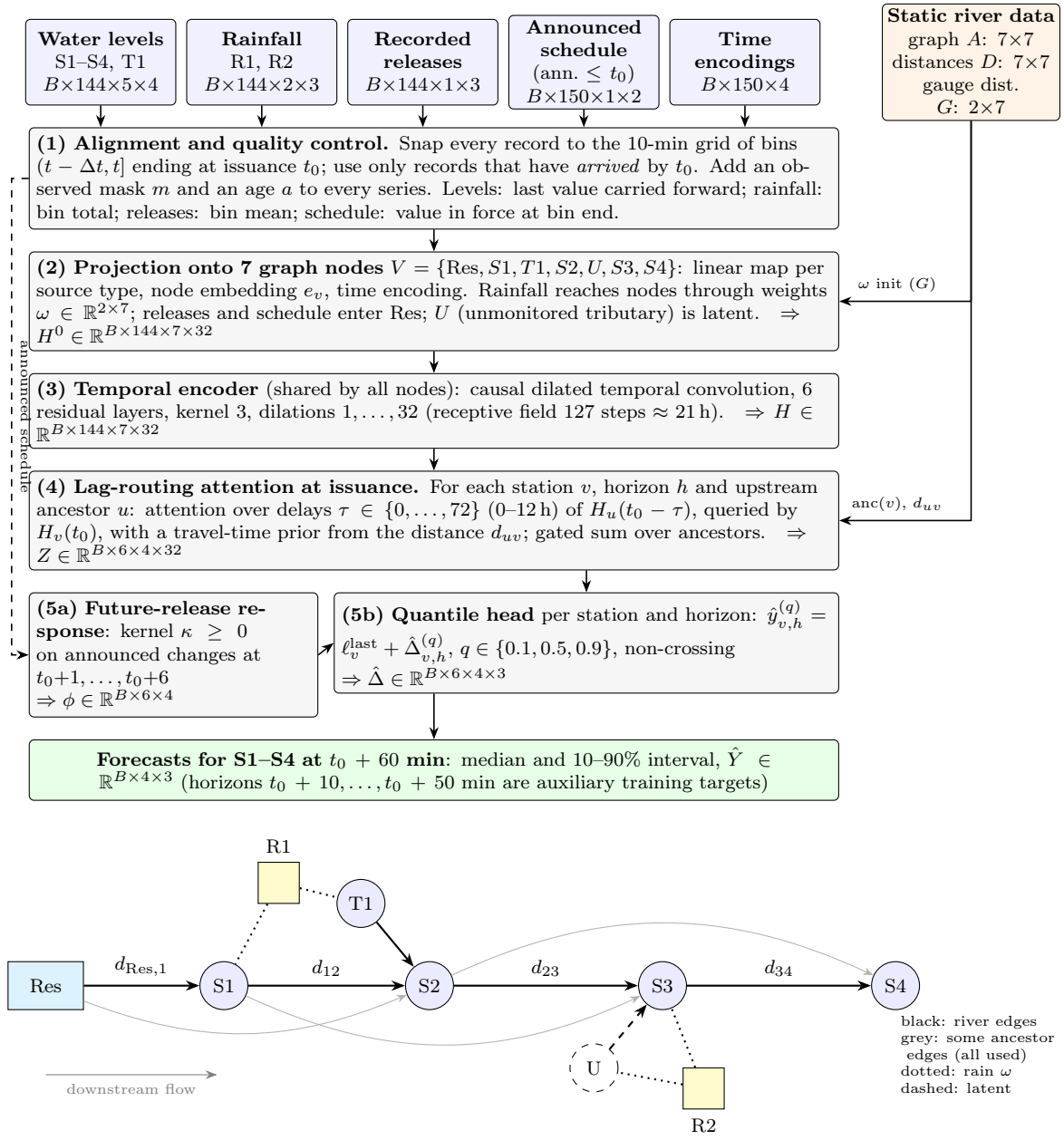


Figure Q1.1: Top: architecture of the Lag-Routing Graph Forecaster, with tensor shapes (B issuance times, 144 history steps of 10 min, 7 nodes, 32 channels, 6 horizons, 3 quantiles). Bottom: the river graph A (schematic; gauge placement as assumed in A5). Each station receives messages from *every* upstream ancestor (transitive closure; a few ancestor edges shown in grey), so every upstream source is one hop away. U is the latent node for the unmonitored tributary; rain gauges feed nodes through the learned weights ω.

2.2 Facts and Assumptions

Supplied facts. F1 levels at S1–S4 and T1 every 10 min, F2 rainfall at R1 and R2 with timestamps and gauge locations, F3 recorded releases, and announced schedules with announcement and effective times, F4 station locations, river connectivity and along-river distances, F5 travel times and unmonitored-tributary inflows are *not* supplied; F6 future rainfall and levels are not available at issuance.

A1 Record length. At least three years of 10-min data with several high-flow events, as is typical of telemetry archives. *Effect:* enough for a model of about 35k parameters; with less data we would lean on the physical priors and the linear baseline.

A2 Units and datum. Levels are in metres on a fixed datum and the stage–discharge relation changes slowly, as is standard for river gauges. *Effect:* levels are predicted directly, normalised per station.

A3 Timestamps and delays. Timestamps are synchronised measurement times (UTC); most records arrive within 0–20 min, with occasional longer outages, as is typical of telemetry. *Effect:* only data that has arrived by t_0 is used, with an age feature, and training simulates the delays.

A4 Missingness. Gaps last minutes to hours and are flagged by the telemetry system, never reported as zero. *Effect:* explicit masks; "no rain" is distinguished from "rain unknown".

A5 Rain gauges. Readings can be totalled per 10-min bin. R1 represents the upper catchment (S1, T1) and R2 the lower catchment, including the unmonitored tributary, since with two gauges each must stand for a region. *Effect:* rainfall reaches nodes through learned weights initialised from gauge distances, so a wrong guess can be corrected in training.

A6 One-way flow. Water flows downstream and backwater is negligible at the stations, as for a reservoir-fed river with no tidal reach. *Effect:* the graph is directed.

A7 Travel times. They grow with distance, shrink as flow rises (kinematic-wave behaviour) and are at most 12 h from the reservoir to S4 (a celerity of about 0.5–3 m/s). *Effect:* delays up to $\tau_{\max} = 72$ steps, a 24-h window, and state-dependent delays.

A8 Reservoir operations. Operators announce at least the next hour of releases at every issuance, recorded releases arrive with the usual delay and can deviate from the schedule in timing or size. *Effect:* separate schedule and release inputs, and a learned response to their difference.

A9 Release is the controlled input. Storage matters downstream only through the release. *Effect:* no reservoir-level input (requested in Table Q1.2).

A10 Stationarity. The catchment response is roughly stationary apart from seasonality, as no major works are reported. *Effect:* time encodings, chronological validation and periodic retraining.

A11 Latent tributary. Its inflow is driven mainly by rainfall in its catchment (best represented by R2), and its past effect shows in S3's history. *Effect:* it is a latent node U fed by rainfall.

A12 As-available data. The telemetry database logs when each value arrived, or this can be reconstructed. *Effect:* training inputs mimic operational ones; otherwise delays are simulated from A3.

2.3 Forecasting task and data specification

Forecasts are issued at every 10-min grid time t_0, only records that have arrived by t_0 are used. The history window is $W = 144$ steps (24 h, $t_0 - 143, \ldots, t_0$), which exceeds the longest assumed travel time (A7) and includes a diurnal cycle. The horizon is $H = 6$ steps; the required output is $h = 6$ ($t_0 + 60$ min) for S1–S4, and $h = 1, \ldots, 5$ are auxiliary training targets.

All series share one clock: a measurement at time s belongs to bin t if $t - \Delta t < s \leq t$ ($\Delta t = 10$ min). Levels take the last valid value in the bin, rainfall is summed, releases are averaged, and the schedule takes the value in force at the bin's end, empty bins carry the previous level forward, or a zero rain value, with mask 0. Across locations, series are *not* pre-shifted by fixed travel times, since these are unknown and vary (F5, A7). Each node keeps its own history;

Table Q1.1: Inputs and outputs for one forecast. B is the batch of issuance times ($B = 1$ in operation). Features: levels [standardised level, 10-min change, mask, age]; rainfall [log(1+bin total), mask, age]; releases [release, mask, age]; schedule [scheduled release, schedule-exists flag].

Tensor	Shape	Axes	Time coverage / timestamps	At issuance
X^{lev}	$B \times 144 \times 5 \times 4$	batch, time, station (S1–S4, T1), feature	$t_0 - 143, \ldots, t_0$; latest values may be stale	yes
X^{rain}	$B \times 144 \times 2 \times 3$	batch, time, gauge (R1, R2), feature	$t_0 - 143, \ldots, t_0$; bin totals	yes
X^{rel}	$B \times 144 \times 1 \times 3$	batch, time, reservoir, feature	$t_0 - 143, \ldots, t_0$; recorded releases	yes
X^{sch}	$B \times 150 \times 1 \times 2$	batch, time, reservoir, feature	$t_0 - 143, \ldots, t_0 + 6$; as known at t_0	yes
X^{time}	$B \times 150 \times 4$	batch, time, sin/cos of day and year	$t_0 - 143, \ldots, t_0 + 6$; from the clock	yes
A, D	7×7 each	node × node	static: edges, river distances	yes
G	2×7	gauge × node	static: gauge–node distances	yes
$\hat{Y}$ (output)	$B \times 6 \times 4 \times 3$	batch, horizon, station, quantile	$t_0 + 10, \ldots, t_0 + 60$ min	–
Y (target)	$B \times 6 \times 4$	batch, horizon, station	observed at $t_0 + 10, \ldots, t_0 + 60$	no
M^Y (mask)	$B \times 6 \times 4$	batch, horizon, station	1 where a target was observed	no

the graph (A, D) and the routing attention decide which past time of each upstream node to read. Rain gauges map onto nodes through ω, reservoir data attach to node Res, and projections bring every source to width $d = 32$ so all nodes stack into one tensor. **Available at issuance:** everything above the line in Table Q1.1; future rainfall, future levels and unannounced releases are not used.

2.4 Model computations and design justification

Notation. t_0 is the issuance step, V the nodes of Figure Q1.1, $\mathcal{S} = \{S1, \ldots, S4\}$ the stations, $\text{anc}(v)$ the nodes with a directed path to v (e.g. $\text{anc}(S4) = \{\text{Res}, S1, T1, S2, U, S3\}$), d_{uv} the along-river distance, $x_v(t)$ the input features of node v, $\ell_v(t)$ its level, and $d = 32$.

(2) Projection. For gauged nodes $h_v^0(t) = W_{\sigma(v)} x_v(t) + e_v + W_c c(t)$, with $\sigma(v)$ the source type and $c(t)$ the time encoding; Res uses $W_q x^{\text{rel}}(t) + W_s x^{\text{sch}}(t)$ in place of $W_{\sigma(v)} x_v(t)$, and U has no observation term. Rainfall is added to every node with non-negative weights:

$$h_v^0(t) \mathrel{+}= \sum_{g \in \{R1, R2\}} \omega_{gv} \, \text{softplus}(W_r) \, x_g^{\text{rain}}(t), \qquad \omega_{gv} = \frac{\exp(-G_{gv}/\lambda + b_{gv})}{\sum_{g'} \exp(-G_{g'v}/\lambda + b_{g'v})}. \tag{1}$$

(3) Temporal encoder. For layers $j = 0, \ldots, 5$ with dilation $\delta_j = 2^j$ (causal, shared across nodes),

$$h_v^{j+1}(t) = h_v^j(t) + W_2^j \, \text{GELU}\Big(\sum_{k=0}^{2} W_{1,k}^j \, h_v^j(t - k\delta_j) \Big), \qquad H_v(t) = h_v^6(t). \tag{2}$$

(4) Lag-routing attention. Water reaching v at $t_0 + h$ left u about μ_{uv} steps earlier, so the relevant state of u lies near $t_0 + h - \mu_{uv}$. For $v \in \mathcal{S}$, horizon h, $u \in \text{anc}(v)$ and delays

$\tau \in \{0, \dots, 72\}$:

$$s_{uv,h}(\tau) = \tfrac{1}{\sqrt{d}}\big(W_Q[H_v(t_0); p_h]\big)^\top W_K[H_u(t_0-\tau); p_\tau] - \frac{\big(\tau - \max(\mu_{uv} - h, 0)\big)^2}{2\sigma_{uv}^2}, \tag{3}$$

$$m_{uv,h} = \sum_\tau \mathrm{softmax}_\tau\big(s_{uv,h}(\tau)\big)\, W_V H_u(t_0-\tau), \qquad \mu_{uv} = \frac{d_{uv}}{\bar{c}\,\Delta t} + \delta\mu_{uv}, \tag{4}$$

$$z_{v,h} = \mathrm{LN}\Big(H_v(t_0) + \sum_{u \in \mathrm{anc}(v)} \gamma_{uv,h}\, m_{uv,h}\Big), \tag{5}$$

with $\bar{c}$ a nominal celerity (A7), learned $\delta\mu_{uv}$ and σ_{uv}, horizon and delay embeddings p_h, p_τ, and sigmoid gates $\gamma_{uv,h}$. This is scaled dot-product attention [9] over delays, as in Autoformer's delay aggregation [10], with a learned counterpart of Muskingum–Cunge routing [11] as prior. Because the query depends on v's current state, the chosen delay can shorten when the river is high.

(5) Forecast mapping. With ℓ_v^{last} the last observed level and $\Delta Q(t_0+k)$ the announced change in release relative to the last recorded release,

$$\phi_{v,h} = \sum_{k=1}^{h} \mathrm{softplus}(\kappa_{v,h-k})\, \Delta Q(t_0+k), \qquad \hat{y}_{v,h}^{(0.5)} = \ell_v^{\mathrm{last}} + w_{v,h}^\top \mathrm{MLP}(z_{v,h}) + \phi_{v,h}, \tag{6}$$

$$\hat{y}_{v,h}^{(0.1)} = \hat{y}_{v,h}^{(0.5)} - \mathrm{softplus}(u_{v,h}^{-\top} z_{v,h}), \qquad \hat{y}_{v,h}^{(0.9)} = \hat{y}_{v,h}^{(0.5)} + \mathrm{softplus}(u_{v,h}^{+\top} z_{v,h}), \tag{7}$$

so a larger announced release can only raise the forecast, and quantiles cannot cross.

Training objective. Masked pinball loss [12]:

$$\mathcal{L} = \frac{1}{B} \sum_b \sum_{v \in \mathcal{S}} \sum_{h=1}^{6} \sum_{q \in \{0.1, 0.5, 0.9\}} \lambda_h\, w(y_{v,h})\, M_{v,h}^Y\, \rho_q\big(y_{v,h} - \hat{y}_{v,h}^{(q)}\big), \qquad \rho_q(e) = \max\big(qe, (q-1)e\big), \tag{8}$$

with $\lambda_6 = 2$, $\lambda_{h<6} = 1$, and $w(y) = 1 + \gamma \mathbb{1}[y > \theta_v]$ up-weighting levels above the alert threshold θ_v (or the 95th percentile). Training uses AdamW, chronological splits by hydrological year, early stopping on validation loss at $h = 6$, and the input corruptions of Section 2.5. The model has about 35k parameters.

How each driver influences each forecast. Upstream conditions reach a station through routing messages from every ancestor at the learned delay. T1 is an observed ancestor of S2–S4; the unmonitored tributary enters S3 and S4 only through the latent node U, built from rainfall (A11). Rainfall enters the nodes it drains to through ω, and its delayed runoff is captured by the encoder and the routing delays Reservoir releases enter twice: past releases are routed from Res at their travel times, and announced changes act through ϕ on the stations near enough to feel them within the hour (in practice S1).

Physical assumptions versus learned mappings. Fixed by physics: the directed topology and ancestor sets (A6), distance-based travel-time priors (A7), non-negative rainfall and release effects, and forecasting the change from the last level. Learned: the actual delays and their dependence on flow, magnitudes, the nonlinear stage response, rainfall–runoff behaviour, the latent tributary and the uncertainty.

Why this design. Attending to all ancestors directly puts every upstream source one hop away, and 72 delays cover the longest travel time; a plain message-passing network [8] would need at least four layers before the reservoir could influence S4. Delay attention rather than fixed shifts, because travel times are unknown and vary and the distance prior makes this learnable from a modest record.A residual from the last observation, because a one-hour change is small relative to the level. *Quantiles,* because flood preparedness needs a credible range. Before deployment the network is compared on validation with persistence and a ridge regression on lagged upstream levels and releases, since simple well-structured models can be hard to beat [13].

2.5 Operation and exceptional conditions

(i) Missing observations. Every input carries a mask and an age, and missing levels are carried forward, so the model knows how stale each value is, missing rain is mask 0, not zero rain. Training randomly masks single readings, short gaps and whole-station outages (about 5–20% of inputs). If a station's own level is missing for over 2 h, its forecast is anchored at the last valid value and labelled "degraded", but a forecast is still issued for all four stations.

(ii) Delayed observations. The system never waits, at each t_0 it uses what has arrived, and the age feature tells the model how old each latest value is. Training reproduces this from as-available snapshots (A12) or simulated delays (A3). A late value is used at the next issuance, issued forecasts are stored unchanged for audit.

(iii) Announced future releases. Only schedule entries announced at or before t_0 are used. They form a step function from $t_0 - 143$ to $t_0 + 6$ that enters node Res and, through ϕ, the near-field forecasts, so a change announced for $t_0 + 30$ min raises S1's forecast immediately. In training the schedule is always the one known at that issuance, never the later actual release, which would be leakage.

(iv) Scheduled versus actual releases. The past uses recorded releases plus the recent deviation $Q^{\mathrm{act}} - Q^{\mathrm{sch}}$, the future uses the schedule. Because training pairs schedules as known with the levels that followed, the model learns typical deviations (e.g. late starts), and its interval widens when the schedule has been unreliable. A deviation beyond a set tolerance is used at the next issuance and logged, if release records are missing, the schedule is used with mask 0.

Feasibility every ten minutes. One forecast processes 144×7 node–time steps at width 32: about 2.5×10^7 multiply-adds for the encoder and about 10^6 for routing, a few milliseconds on one CPU core, well within the 10-min cycle. Retraining is offline (e.g. seasonally), and a persistence-plus-ridge fallback is published if the network or a data feed fails.

2.6 Uncertainty, limitations, and additional information

- **Unmonitored tributary.** Its inflow between S2 and S3 is unobserved, so S3 and S4 areless certain after local storms. *Response:* latent node U driven by R2, S3's own rise, and wider intervals at S3/S4. *Unresolved:* storms that R2 does not register.
- **Changing travel times.** Rare regimes such as overbank flow are under-represented and cause peak-timing errors. *Response:* state-dependent delays, up-weighted high levels, and flags when inputs leave the training range.
- **Travel times shorter than the horizon.** The needed upstream state is still in the future, so the model extrapolates the upstream trend, errors grow on fast rises and the interval reflects this.
- **No future rainfall (F6).** Rain in the next hour can raise S3/S4 unexpectedly. *Response:* wider intervals in rainy periods; *unresolved* without nowcasts.
- **Rare extremes and unannounced releases.** Peaks may be under-predicted, and emergency releases are unseen until recorded. *Response:* loss weighting, monotone priors, and reflection within one cycle once a release is recorded.
- **Sensors, gauge coverage and non-stationarity.** Drift, two gauges for the whole basin (A5), and channel or land-use change. *Response:* quality-control flags, learned ω, monitoring of skill against persistence, and retraining; gradual drift may go undetected.

Table Q1.2: Additional information to request from the authority.

Request	How it helps	Usable without it?
Rainfall nowcasts or radar	Rain for the next hour; fewer surprise rises at S3/S4	Yes, wider intervals
Gauge on the unmonitored tributary	Turns U into an observed node; largest gain for S3/S4	Yes, U stays latent
Alert thresholds, rating curves, cross-sections	Sets θ_v; enables discharge-based checks	Yes, percentile proxies
Travel-time studies or a hydraulic model	Better priors μ_{uv}; synthetic extreme events	Yes, distance priors
History of schedules vs actual releases	Better model of deviations and their timing	Yes, learned from records
Telemetry arrival logs, maintenance records	Realistic delay simulation; excludes faulty periods	Yes, assumed delays
Catchment boundaries, elevation, land use	Better rainfall-to-node mapping ω	Yes, distance initialisation
Reservoir level and inflow	Anticipates forced releases near full capacity	Yes

3 Question 2: Motion or Memory?

3.1 Setup

Moving MNIST [1] is loaded with `split=None` and split by dataset index (0–7,999 train, 8,000–8,999 validation, 9,000–9,999 test). The frames are divided by 255, and targets beyond frame 20 are masked out of every loss and metric. All code is in the `pa2` package and runs from five Colab notebooks on a Tesla T4. The checks in Table 1 call the same window and transformation functions as the experiments. Training time per epoch counts training batches only. Inference time covers the 1,000 test videos at batch 32 under `eval()` and `no_grad()`, from when each batch is on the GPU, after one untimed warm-up batch.

Table 1: Implementation checks on synthetic tensors (x: random $20 \times 1 \times 64 \times 64$, p: zeros with $p[7, 0, 10, 20] = 1$; window checks on a video whose frame f is filled with f).

	Check	Expected	Ours
(a)	$\max \lvert M(M(x)) - x \rvert$	0	0 (`torch.equal`)
(b)	$\max \lvert R^4(x) - x \rvert$	0	0 (`torch.equal`)
(c)	$\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	0 (`torch.equal`)
(d)	bright pixel of $M(p)$ (frame, row, col.)	(7, 10, 43)	(7, 10, 43)
(e)	bright pixel of $R(p)$ (frame, row, col.)	(7, 43, 10)	(7, 43, 10)
(f)	1× input / target frames	1–5 / 6–10	1–5 / 6–10
(g)	2× input / target frames	1,3,5,7,9 / 11,13,15,17,19	1,3,5,7,9 / 11,13,15,17,19
(h)	3× input / target frames	1,4,7,10,13 / 16,19	1,4,7,10,13 / 16,19

Q2.1 Figure 1(a) is not flat, the bins range from 2.6% to 8.2% against a uniform 5.6%, and up-left motion (120°–180°) is the rarest. Every direction is nevertheless well represented, and this unevenness is expected from only about 100 digit trajectories. A mirror maps θ to $180° - \theta$ and a 90° rotation maps it to $\theta + 90°$, so both only reshuffle the bins. The reshuffled histograms differ from the original about as much as the original differs from uniform (total-variation distance 0.13 and 0.16, against 0.11), and wall bounces reflect trajectories anyway. Mirrored and rotated motion is therefore familiar, only the digits' appearance is new. From Figure 1(b) the median speed is 3.32 px/frame (most pixels 2.5–4.5), so 2× corresponds to about 6.6 and 3× to about 10 px/frame, speeds absent from training. The sharp peaks near 3.16, 3.61 and 4.12 px are the le ngths of the integer displacements (3,1), (3,2) and (4,1), so the flow is accurate where digits are separate. Overlap averages two digits' motion inside Farnebäck's window, biasing

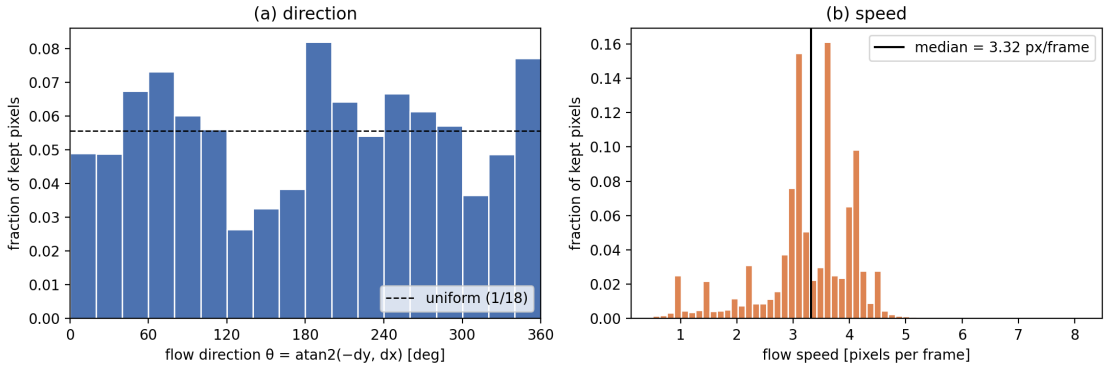


Figure 1: Farnebäck optical flow [3] on the first 50 training videos (950 frame pairs; handout settings); 103,050 pixels are kept. (a) Direction $\theta = \text{atan2}(-d_y, d_x)$ in 18 bins of 20°, as fractions of kept pixels; dashed line: uniform level 1/18. (b) Speed; the median is 3.32 px/frame.

those pixels towards low speeds and intermediate directions (probably part of the bump near 1 px/frame). Overlap pixels are a small share, however, and averaging smooths a histogram rather than emptying a direction range, so neither answer changes.

Q2.2 Mirror and rotation change appearance (mirror-image and sideways digits never occur in MNIST, and rotation changes them more) while keeping every speed and only remapping directions, which by Q2.1 leaves the motion statistics essentially unchanged. Speed-up does the reverse. The digits look as in training, but every displacement is multiplied by k, beyond the training range. Each transformation thus changes one factor while holding the other in distribution, all three obey the same physics (square box, constant velocity, elastic bounces), and the correct answer is known exactly ($T(Y)$ for the pixel permutations M and R, real frames for $k\times$).

3.2 Baseline ConvLSTM

Model A is implemented exactly as specified (`pixel_unshuffle`(4), two ConvLSTM layers [2] with 64 channels and 3×3 kernels, 1×1 output convolution, `pixel_shuffle` and sigmoid, 9-step rollout without teacher forcing) and has 480,784 trainable parameters.

Table 2: Test split, $1\times$, $t = 1$. Learned model: mean (seed 0, seed 1).

Model	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	$E^{(5)}$	Params
A	0.0171 (0.0173, 0.0169)	0.0205 (0.0208, 0.0201)	0.0227 (0.0231, 0.0223)	0.0245 (0.0248, 0.0241)	0.0261 (0.0264, 0.0259)	0.0222 (0.0225, 0.0219)	480,784
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

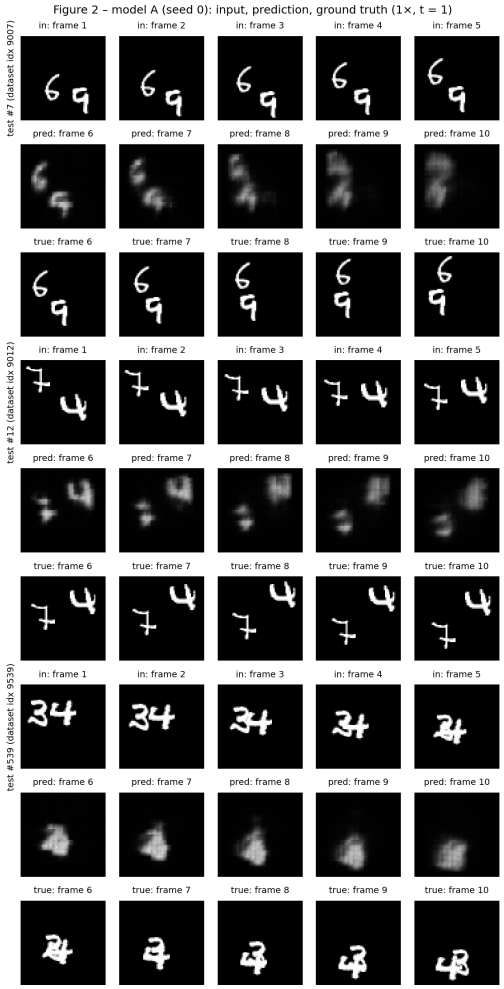


Figure 2: Seed-0 model A on three test videos (1×, $t = 1$): input frames 1–5, predicted and true frames 6–10. Tests #7 and #12 (dataset indices 9007, 9012) have no overlap in the target frames; in test #539 (dataset index 9539) the digits overlap by 33–87 pixels in frames 6–10.

Q2.3 `pixel_unshuffle`(4) makes one cell step equal to 4 px, centre to centre. Within one step τ, layer 1's single 3×3 convolution over $[u_\tau, h^{(1)}_{\tau-1}]$ moves information at most ± 1 cell, the cell-state update is element-wise and moves nothing; layer 2's 3×3 convolution over $[h^{(1)}_\tau, h^{(2)}_{\tau-1}]$ adds ± 1 cell. And the 1×1 convolution, `pixel_shuffle` and sigmoid add nothing. The paths $u_\tau \to$ layer 1 $\to$ layer 2 $\to \hat{x}_{\tau+1}$ and $h^{(1)}_{\tau-1} \to$ layer 1 $\to$ layer 2 span 2 cells, and $h^{(2)}_{\tau-1} \to$ layer 2 spans 1. Model A's reach is therefore **2 cells = 8 px** along a row or column ($2\sqrt{2}$ cells $\approx$ 11.3 px diagonally). The recurrence does not raise it: information spreads $8n$ px over n steps, but a single update connects positions at most 8 px apart.

Q2.4 Model A beats both references at every horizon. $E^{(5)} = 0.0222$ is about half of Blank's 0.0426 and a third of Copy-last's 0.0641, with the seeds within 3%. Copy-last is worse than Blank even at $h = 1$ (0.0492 against 0.0426): after a 3–4 px move a copied digit is penalised twice, bright where the target is dark and dark where it is bright. Model A's error grows from 0.0171 to 0.0261 (+53%) in shrinking steps, while Blank is flat and Copy-last rises with the accumulated displacement. Figure 2 shows why? Frame 6 is well placed, but every prediction is fed back as input, so blur compounds until the digits of tests #7 and #12 are grey blobs near their true positions by frame 10. Under squared error, a blurred average over plausible positions costs less than a sharp guess in the wrong place. With overlap the failure differs, in test #539 the 3 and 4 merge into one fading smudge, whereas in truth they pass through each other and

separate by frame 10. Once digits share pixels, the model loses track of which pixels belong to which digit.

Q2.5 — prediction written and committed before any "Testing the Baseline" experiment (`predictions/Q2_5.md`)– **Mirror test.** I expect model A to mostly pass. Figure 1(a) shows that training motion already covers all directions, so mirrored trajectories are familiar motion. Only the mirrored digit shapes are new. A ConvLSTM's 3×3 kernels are not constrained to be symmetric, so I do not expect exact consistency: C_M will be clearly above zero. I do expect it to be small compared with E(5), and E_M to be only slightly higher than E(5).

Rotation test. I also expect model A to mostly pass, but less cleanly than the mirror test. Rotated directions are equally familiar ($\theta \to \theta + 90°$). However, a 90° rotation turns the mostly vertical strokes of MNIST digits into horizontal ones, a bigger change in local appearance than mirroring. I therefore predict C_R > C_M and E_R > E_M, with neither gap becoming catastrophic.

Speed. My reach estimate for model A is 8 px per step along rows and columns (≈11 px along a diagonal). The typical speed in Figure 1(b) is about 3.3 px/frame, so 2× is about 6.6 px/frame and 3× about 10 px/frame.

- **At 2×**, most displacements are still inside the reach, so the model should keep tracking the digits. Its error will rise noticeably, because it has never seen this speed and will probably move the digits too slowly. The fastest digits (about 9 px/frame at 2×) are already beyond 8 px.
- **At 3×**, the typical displacement exceeds the reach, so the model cannot connect a digit's old position with its new one in a single update. I expect it to break down here: predictions will lag behind and blur into smears, with E(2) at 3× approaching the Blank level.

3.3 Testing the Baseline

Table 3: Consistency gaps and transformed errors (test split, 1×, $t = 1$, averaged over the five horizons). Learned model: mean (seed 0, seed 1).

Model	$E^{(5)}$	C_M	E_M	C_R	E_R
A	0.0222 (0.0225, 0.0219)	0.00199 (0.00176, 0.00222)	0.0228 (0.0231, 0.0226)	0.00322 (0.00280, 0.00364)	0.0239 (0.0240, 0.0238)
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641

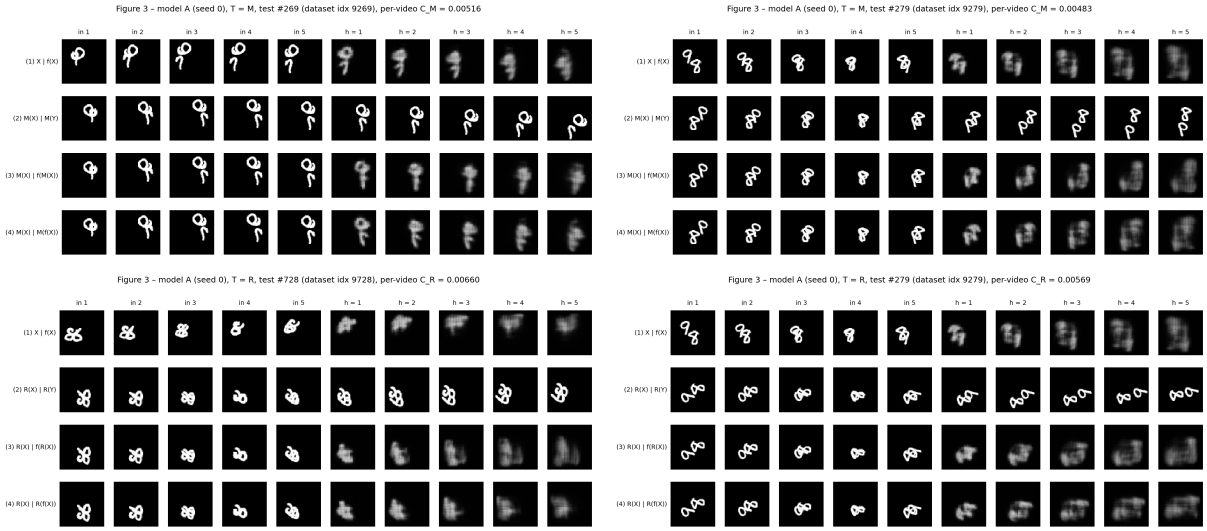


Figure 3: Seed-0 model A on the two test videos with the largest per-video C_M (top: tests #269, #279) and the two with the largest per-video C_R (bottom: tests #728, #279); test #i is dataset index $9000 + i$. Rows: (1) $X \,|\, f(X)$; (2) $T(X) \,|\, T(Y)$; (3) $T(X) \,|\, f(T(X))$; (4) $T(X) \,|\, T(f(X))$. GIFs: `gifs/fig3_M_rank1_test269.gif`, `gifs/fig3_M_rank2_test279.gif`, `gifs/fig3_R_rank1_test728.gif`, `gifs/fig3_R_rank2_test279.gif`.

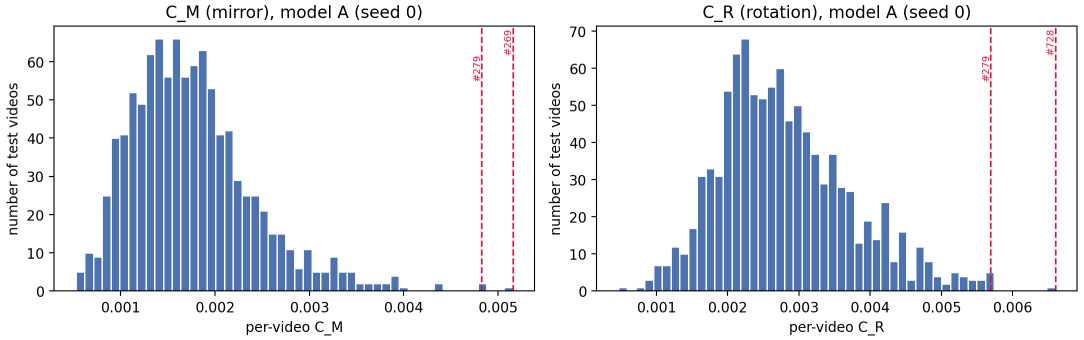


Figure 4: Per-video C_M (left) and C_R (right) of seed-0 model A over the 1,000 test videos. Dashed red lines mark the Figure 3 videos.

Q2.6 Blank and Copy-last get $C_M = C_R = 0$ exactly, because their outputs commute with any pixel permutation, yet they are the worst predictors in Table 3 ($E_M = E_R = 0.0426$ and 0.0641, against 0.0228 and 0.0239 for model A). A small C_T only says that the model treats $T(X)$ the same way as X, not that the treatment is right: a model can be perfectly consistent by ignoring its input. E_T shows whether the consistent prediction is also correct. Read together, small C_T with small E_T means a rule that carries over to the transformed scene; small C_T with large E_T means consistently wrong, like Blank; large C_T means the transformed scene is handled differently.

Q2.7 In test #728 ($T = R$, the largest per-video C_R, 0.00660), an 8 and a 6 overlap in all five input frames and, in truth (row 2), move together up and to the left. Predicting from the rotated input (row 3), the model sends the pair upward but drifts it to the right, and by $h = 5$ the blob has split into two faint side-by-side lobes, as if the digits separated sideways. Predicting from the original and rotating back (row 4), the pair barely moves and fades into a wide smear near its last position. In one orientation the digits rise and come apart and in the other they stall and dissolve, and neither follows the true path. All four Figure 3 videos have overlapping inputs,

which suggests that the inconsistency concentrates where the model must tell digits apart by shape.

Q2.8 Yes, in both seeds: $C_R = 0.00322$ is $1.6\times$ $C_M = 0.00199$, and E_R rises 7.7% above $E^{(5)}$ against 2.7% for E_M. Figure 4 shows the whole C_R distribution shifted (most videos at 0.002–0.003 against 0.001–0.002 for C_M), not a few outliers. Motion cannot explain this as both transformations keep every speed and only remap directions, which cover the full circle (Q2.1), so rotated motion is as familiar as mirrored motion. The cause is appearance. A mirror keeps vertical strokes vertical and only flips their slant, whereas a 90° rotation makes the mostly vertical strokes of MNIST digits horizontal, a larger shift in the local patterns the 3×3 kernels have learned. This fits the worst cases being overlapping digits that must be kept apart by shape.

Table 4: Faster motion (test split, $t = 1$). Only $E^{(2)}$ is comparable across speeds, because $3\times$ has only two target frames. Learned model: mean (seed 0, seed 1).

Model	$E^{(2)}$ $1\times$	$E^{(2)}$ $2\times$	$E^{(2)}$ $3\times$	$E^{(5)}$ $1\times$	$E^{(5)}$ $2\times$
A	0.0188 (0.0190, 0.0185)	0.0453 (0.0443, 0.0463)	0.0530 (0.0524, 0.0536)	0.0222 (0.0225, 0.0219)	0.0506 (0.0482, 0.0531)
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

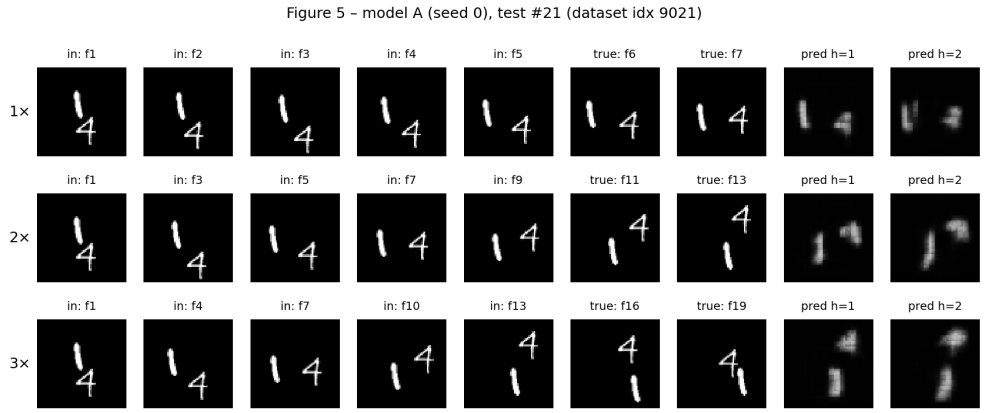


Figure 5: Test #21 (dataset index 9021), seed-0 model A: the five input frames and the first two true and predicted future frames at $1\times$, $2\times$ and $3\times$ ($t = 1$). Its digits do not overlap in any of the displayed true frames.

Q2.9 Model A falls apart already at $2\times$, not at $3\times$ as predicted in Q2.5: $E^{(2)}$ rises from 0.0188 to 0.0453 at $2\times$ and 0.0530 at $3\times$. Blank's error is the targets' mean squared intensity (0.0426) and barely depends on speed, so at $2\times$ model A is at about the Blank level and at $3\times$ clearly worse; its advantage over Copy-last shrinks from 66% to 31% and 25%. This disagrees with the reach estimate, since 6.6 px is inside 8 px. Reach is only an upper bound on how far information can travel in one step: being within reach is necessary but not sufficient, and the estimate missed that the model must also have learned a rule for that speed. Model A has only seen about 3.3 px/frame (and model C, with the same reach, later cuts $E^{(2)}$ at $2\times$ by 37%).

In Figure 5, the $1\times$ predictions are in the right places, slightly blurred. At $2\times$ *the digits lag*: they move the right way but too slowly, staying near their frame-9 positions, with the 1 smeared into a vertical streak and the 4 reduced to a shapeless blob. At $3\times$ *they stall* at their frame-13 positions and fade, while in truth both bounce off the walls. Nothing is duplicated and nothing vanishes outright. Lagging and stalling suggest that the model does not measure speed from its inputs but applies at most its $1\times$ step. the streak and the blob are the hedge squared error

rewards when unsure where along the path a digit is and the missed bounces show that it does not anticipate collisions reached faster than in training.

Q2.10 No, at 2× and 3× the motion is both unfamiliar and, at 3×, beyond the reach, and both explanations grow with speed. To separate them, change one factor at a time. Train model A's architecture with speed augmentation (model C): if C succeeds at 3×, the failure was unfamiliarity. Then keep the augmentation and enlarge the reach (model D): if C still fails at 3× while D succeeds, the limit is architectural. If reach binds, C's gain should be large at 2× and small at 3×, and D's advantage should appear mainly at 3×.

3.4 What Can Data Fix?

Models B and C each change one thing relative to A (symmetry augmentation with g uniform on G; speed augmentation with k uniform on $\{1, 2, 3\}$), and D changes only the architecture relative to C (Q2.12).

Q2.11 — prediction written and committed before training models B–D (`predictions/Q2_11.md`); reproduced unedited. Model B (symmetry augmentation) will shrink both consistency gaps, C_R more than C_M because rotated digits were the least familiar appearance, but it will not help at 2× or 3×. Model C (speed augmentation) will clearly improve 2× but stay poor at 3×, because about 10 px/frame is beyond the reach of model A's architecture whatever it is trained on. Model D (larger reach, same speed augmentation) will be the only model that also fixes 3×, possibly at a small cost at 1×, while leaving the mirror and rotation gaps roughly where model A has them.

Table 5: Models A–D on the test split ($t = 1$). Learned models: mean (seed 0, seed 1). Timings on a Tesla T4 (PyTorch 2.11.0+cu130); training time is the mean wall-clock time per epoch over 30 epochs (training batches only); inference covers the 1,000 original test videos at batch 32.

	A	B	C	D
$E^{(5)}$ 1×	0.0222 (0.0225, 0.0219)	0.0217 (0.0223, 0.0211)	0.0265 (0.0262, 0.0267)	0.0273 (0.0273, 0.0273)
C_M	0.00199 (0.00176, 0.00222)	0.00131 (0.00123, 0.00138)	0.00144 (0.00147, 0.00142)	7.71×10^{-4} (8.72×10^{-4}, 6.69×10^{-4})
E_M	0.0228 (0.0231, 0.0226)	0.0217 (0.0223, 0.0211)	0.0267 (0.0265, 0.0270)	0.0275 (0.0275, 0.0275)
C_R	0.00322 (0.00280, 0.00364)	0.00158 (0.00153, 0.00164)	0.00277 (0.00250, 0.00304)	0.00184 (0.00175, 0.00193)
E_R	0.0239 (0.0240, 0.0238)	0.0218 (0.0223, 0.0213)	0.0277 (0.0274, 0.0280)	0.0284 (0.0283, 0.0284)
$E^{(2)}$ 1×	0.0188 (0.0190, 0.0185)	0.0182 (0.0188, 0.0177)	0.0232 (0.0231, 0.0232)	0.0245 (0.0245, 0.0246)
$E^{(2)}$ 2×	0.0453 (0.0443, 0.0463)	0.0435 (0.0429, 0.0442)	0.0286 (0.0287, 0.0285)	0.0290 (0.0287, 0.0293)
$E^{(2)}$ 3×	0.0530 (0.0524, 0.0536)	0.0473 (0.0461, 0.0484)	0.0333 (0.0335, 0.0331)	0.0327 (0.0325, 0.0330)
Params	480,784	480,784	480,784	480,784
Train s/epoch	10.7 (10.4, 10.9)	11.5 (11.5, 11.5)	10.9 (10.9, 10.9)	16.3 (16.3, 16.3)
Inference s	0.465 (0.460, 0.471)	0.433 (0.431, 0.435)	0.440 (0.438, 0.442)	0.708 (0.706, 0.709)

Q2.12 Model D is model A with one change: layer 2's 3×3 convolution is dilated by 2 [4], so its taps sit at cell offsets $-2, 0, +2$. Everything else is identical, and dilation adds no weights, so D has 480,784 parameters (limit 721,176). With the conventions of Q2.3, layer 1 moves information ±1 cell and layer 2 ±2 cells, so the paths $u_\tau \to$ layer 1 $\to$ layer 2 and $h^{(1)}_{\tau-1} \to$

layer 1 $\to$ layer 2 span 3 cells, and $h^{(2)}_{\tau-1}$ $\to$ layer 2 spans 2. D's reach is **3 cells = 12 px** ($\approx$17 px diagonally), strictly larger than A's 8 px and above the typical 3$\times$ displacement; the offsets $\{-1,0,1\}+\{-2,0,2\}=\{-3,\dots,3\}$ leave no holes. D was chosen on the validation split from three candidates trained for 10 epochs (5×5 kernels with 46 channels, this design, and 8×8 patches). Only the dilated design learned (validation $E^{(2)}$ 0.0300, 0.0320 and 0.0344 at 1$\times$, 2$\times$ and 3$\times$); the other two stayed on the near-blank plateau (about 0.039 at every speed), so the choice reflects how quickly a design starts to learn as much as its final quality.

Q2.13 Mirror and rotation gaps $\to$ appearance (A vs B). Changing only the digit orientations seen in training cuts C_M by 34% and C_R by 51%, and brings E_M and E_R down to B's own $E^{(5)}$ (0.0217); B helps far less at 2$\times$ and 3$\times$ (-4% and -11%). C and D also have lower C_M, but with higher E_M: blurrier predictions agree more, the caveat of Q2.6. Only B is both more consistent and no worse on transformed clips.

2$\times$ failure $\to$ unfamiliar motion (A vs C). With the same architecture, speed augmentation lowers $E^{(2)}$ at 2$\times$ from 0.0453 to 0.0286 (-37%).

3$\times$ failure $\to$ mainly unfamiliar motion as well (A vs C vs D). C, with A's 8 px reach, already cuts $E^{(2)}$ at 3$\times$ from 0.0530 to 0.0333 (-37%); D's 12 px reach adds only 2% (0.0327), about the seed spread. The scenario in the question, C bad at 3$\times$ and D good, did not occur. Had it occurred, the failure would be architectural and more data could not fix it. Training changes only the weights, not the computation graph, and within one step the output at a pixel depends only on inputs and states within the reach, so no weights can make it use pixels outside. C plausibly copes because a 20-px digit moved by 10 px still lies partly within 8 px of its old extent, and the recurrent state carries motion across steps. Both C and D still degrade from 1$\times$ to 3$\times$ (+44% and +33%); with only two targets per 3$\times$ window, 3$\times$ is also the least-trained case.

Q2.14 B costs nothing at 1$\times$ ($E^{(5)}$ 0.0217 against 0.0222). C and D do: $E^{(5)}$ at 1$\times$ rises by 19% and 23%, and $E^{(2)}$ by 23% and 30%, because the same capacity now covers three speeds and 1$\times$ predictions become blurrier. In exchange, $E^{(2)}$ falls by 36–38% at 2$\times$ and 3$\times$; averaged over the three speeds it is 0.0390 for A, 0.0363 for B, 0.0284 for C and 0.0287 for D. If test speeds vary, speed augmentation is worth its cost, if only 1$\times$ matters, it is not. D's larger reach is not worth it here: 52% more training and inference time, and a slightly worse 1$\times$ error than C, for a 2% gain at 3$\times$.

3.5 The Final Model

For any trained f, the symmetrised forecaster is $\bar{f}(X)=\frac{1}{8}\sum_{g\in G} g^{-1}\big(f(g(X))\big)$ [5]. Our final model is symmetrised D. We did not train D+B: B's main contribution in Table 5 is consistency, which symmetrisation already provides exactly, and B's accuracy gains over A were modest ($E^{(2)}$ -3%, -4% and -11% at 1$\times$, 2$\times$ and 3$\times$), so D+B might be somewhat more accurate than D, especially at 3$\times$, but we did not test this.

Table 6: Symmetrised models on the test split ($t = 1$). Training time is that of the underlying model; inference includes all 8 transformations, forward passes, inverse transformations and the averaging. Same hardware as Table 5.

	symmetrised B	symmetrised D (final)
$E^{(5)}$ 1×	0.0211 (0.0217, 0.0205)	0.0273 (0.0272, 0.0273)
C_M	8.56×10^{-17} (7.95×10^{-17}, 9.17×10^{-17})	6.12×10^{-17} (6.02×10^{-17}, 6.21×10^{-17})
E_M	0.0211 (0.0217, 0.0205)	0.0273 (0.0272, 0.0273)
C_R	6.89×10^{-17} (6.42×10^{-17}, 7.36×10^{-17})	4.71×10^{-17} (4.62×10^{-17}, 4.80×10^{-17})
E_R	0.0211 (0.0217, 0.0205)	0.0273 (0.0272, 0.0273)
$E^{(2)}$ 1×	0.0178 (0.0183, 0.0173)	0.0246 (0.0246, 0.0246)
$E^{(2)}$ 2×	0.0427 (0.0419, 0.0434)	0.0290 (0.0287, 0.0293)
$E^{(2)}$ 3×	0.0460 (0.0448, 0.0472)	0.0327 (0.0324, 0.0329)
Params	480,784	480,784
Train s/epoch	11.5 (11.5, 11.5)	16.3 (16.3, 16.3)
Inference s	3.40 (3.39, 3.41)	5.57 (5.61, 5.53)

Q2.15 Consistency. Learned symmetry (B) reduces the gaps (C_M 0.00131 and C_R 0.00158, against 0.00199 and 0.00322 for A) but does not remove them. Built-in symmetry makes them zero up to rounding ($\sim 10^{-17}$, Table 6), and exactly so for any f, because the average runs over the whole group. For any $h \in G$,

$$\bar{f}(h(X)) = \frac{1}{8}\sum_{g\in G} g^{-1} f(ghX) \overset{g'=gh}{=} \frac{1}{8}\sum_{g'\in G} (g'h^{-1})^{-1} f(g'X) = \frac{1}{8}\sum_{g'\in G} h\, g'^{-1} f(g'X) = h\big(\bar{f}(X)\big), \tag{9}$$

since $g \mapsto gh$ is a bijection of G and h, a pixel permutation, commutes with averaging. Hence $\bar{f} \circ M = M \circ \bar{f}$ and $\bar{f} \circ R = R \circ \bar{f}$, so $C_M = C_R = 0$; the residual $\sim 10^{-17}$ is floating-point rounding. B's symmetry is only a training incentive, which its 3×3 kernels satisfy approximately.

Accuracy. On original clips, symmetrised B is slightly better than B (0.0211 against 0.0217) and symmetrised D equals D (0.0273). On transformed clips, built-in symmetry gives $E_M = E_R = E^{(5)}$ exactly; B comes close (0.0217, 0.0218). At 2× and 3× symmetrisation changes little. **Inference cost.** Eight forward passes make built-in symmetry about 7.9× slower (3.40 s against 0.433 s for B; 5.57 s against 0.708 s for D); learned symmetry costs nothing at inference, only slightly longer training (11.5 against 10.7 s/epoch). **Blur.** The average of eight predictions is sharp only where they agree. Its error is at most the mean error of the eight transformed predictions, so averaging loses accuracy only when f is clearly worse or inconsistent on transformed inputs. It did not hurt here, because B and D are already fairly consistent, so averaging acts like a small ensemble (symmetrised B even gains 3%), though Figure 6 shows the averaged predictions as smooth blobs.

Q2.16 Partly. **The disagreement is gone:** for symmetrised D, rows 3 and 4 of Figure 6(a) are identical for test #728 (per-video $C_R = 9.85\times10^{-17}$ against 0.00660 for model A), fixed by construction. **The motion is still wrong, and the appearance is worse.** In truth the cluster moves up and to the left; the final model predicts a featureless grey oval that stays near the digits' last input position and grows and fades. Neither digit is recognisable even at

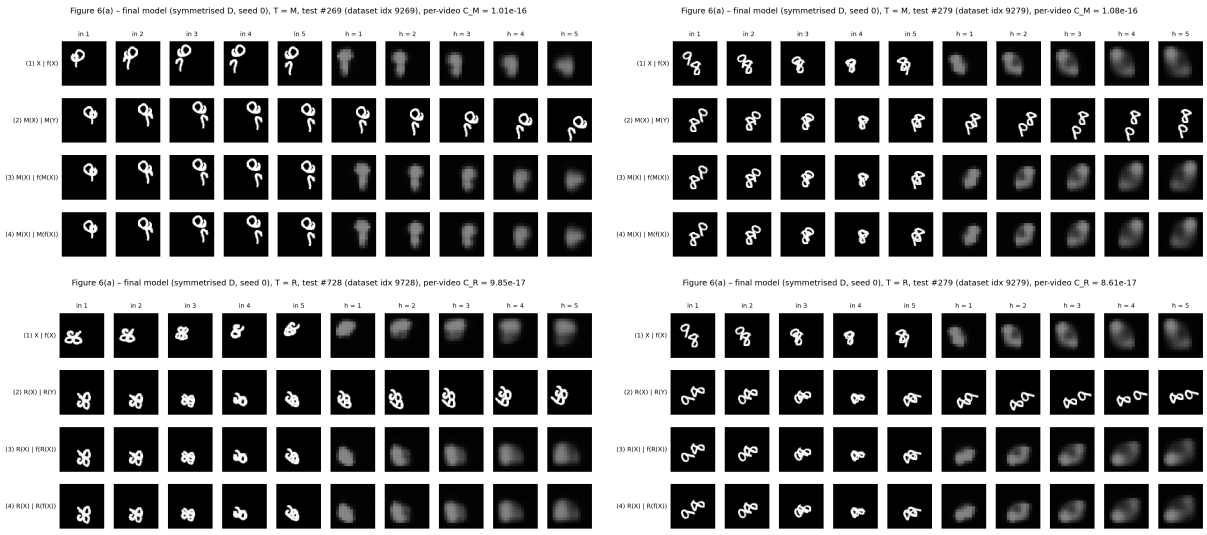


Figure 6(b) – final model (symmetrised D, seed 0), test #21 (dataset idx 9021)

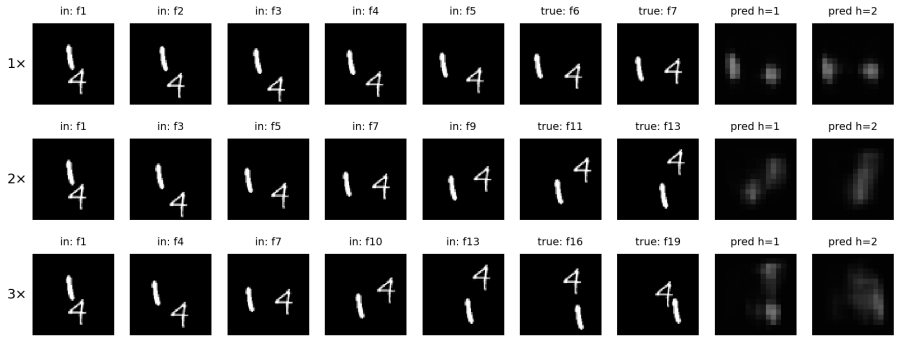


Figure 6: Final model (symmetrised D, seed 0). (a) Top four panels: Figure 3 repeated on exactly the same videos and transformations, rows as in Figure 3; GIFs: `gifs/fig6a_M_rank1_test269.gif`, `gifs/fig6a_M_rank2_test279.gif`, `gifs/fig6a_R_rank1_test728.gif`, `gifs/fig6a_R_rank2_test279.gif`. (b) Bottom: Figure 5 repeated on the same video (test #21) at 1×, 2× and 3×.

$h = 1$, whereas model A kept some structure early on. **Why:** speed augmentation and D's larger reach do nothing for a 1× clip whose difficulty is two overlapping digits, and D's blurrier 1× predictions ($E^{(5)}$ 0.0273 against 0.0222) show up as lost digit shape. On this clip the final model is consistent, but consistently vague.

Q2.17 Partly. The evidence that it learned motion rather than memorising appearance is threefold. Its predictions are exactly consistent under mirror and rotation. It beats Copy-last by more than half at every speed. And speed augmentation cut $E^{(2)}$ at 2× and 3× by more than a third without changing the architecture, so the motion rule now covers speeds model A could not follow. A skeptic would still want more: forecasts that keep both digits sharp and correctly placed over all five horizons, and generalisation beyond the training distribution, such as 4× motion, unseen digits or three digits at once. One test it still fails is overlapping digits. When two digits cross, as in test #728, it merges them into one blob and loses their identities and separate directions. Its 1× accuracy ($E^{(5)}$ 0.0273) is also worse than model A's (0.0222): it has traded sharpness for coverage.

4 Conclusion and Limitations

Both questions turned on whether information can travel far enough, and on whether the model has learned to use the paths it has. The river forecaster puts every upstream source one hop from every station and learns the delay at which to read it around a physical travel-time prior. In Question 2, reach predicted which designs could follow faster motion, but it was not the binding constraint: model A failed at 2×, well within its reach, because it had never seen that speed, and speed augmentation alone recovered most of the loss. Augmentation reduced the symmetry gaps, and building symmetry into the forecaster removed them exactly. **Limitations:** Q1 is an untested design whose assumptions (especially travel times and gauge representativeness) must be checked against real data, each model has only two seeds, so few-percent differences such as D against C are at the level of seed spread; D was chosen from 10-epoch runs in which two candidates had not started to learn; Blank was evaluated only at 1×; and D+B was not trained.

References

[1] N. Srivastava, E. Mansimov, and R. Salakhutdinov, "Unsupervised learning of video representations using LSTMs," in *Proc. ICML*, 2015.

[2] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-C. Woo, "Convolutional LSTM network: A machine learning approach for precipitation nowcasting," in *Proc. NeurIPS*, 2015.

[3] G. Farnebäck, "Two-frame motion estimation based on polynomial expansion," in *Proc. Scandinavian Conference on Image Analysis (SCIA)*, 2003.

[4] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," in *Proc. ICLR*, 2016.

[5] T. Cohen and M. Welling, "Group equivariant convolutional networks," in *Proc. ICML*, 2016.

[6] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. ICLR*, 2018.

[7] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. IJCAI*, 2018.

[8] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. ICML*, 2017.

[9] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[10] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. NeurIPS*, 2021.

[11] J. A. Cunge, "On the subject of a flood propagation computation method (Muskingum method)," *J. Hydraulic Research*, vol. 7, no. 2, pp. 205–230, 1969.

[12] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.

[13] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *Proc. AAAI*, 2023.

5 AI Policy Compliance

Please note that claude and gemini was used for help in coding, formatting i.e. making tables and converting solutions into latex format and understanding of the underlying concepts

- **Prompt:** Can you explain the forecasting setup in Question 1 in simple terms. What exactly is predicted, from what, and when?
  AI output: Explained the complete forecasting setup with the example and the intuitive explanation and why it is forward prediction approach.
- **Prompt:** Can you briefly explain ConvLSTM, how do they work and why are they used in this assignment specifically? also give me the intuitive explanation with an example.
  AI output: Explained the concept ConvLSTM and their internal working. Also provided the visual representations of the ConvLSTM.
- **Prompt:** Model A fails at $2\times$ even though $2\times$ is within its reach. What can be the possible explanation?
  AI output: Suggested three candidates: unfamiliar speed, an effective reach smaller than the theoretical one, and blur from feeding predictions back as input. *Edits* I kept the unfamiliar-speed explanation because model C confirmed it. I dropped one argument the AI proposed ("compare speed with half the reach") because model C's results contradicted it.
- **Prompt:** How does a mirror change a flow direction , and how does a 90° rotation change it? Check my mapping
  AI output: Checked it, provided the explanation and that mapping was right.
- **Prompt:** what is causing the speed-up change is it just motion only as it doesnot make sense to me, or does it also change what the digits look like in some way
  AI output and corrections: Provided the details that Each individual frame is unchanged by speed-up. What changes is how the frames relate to each other, so in that sense it really is a motion-only change. But "motion" changes in more ways than just speed,